

CS 3513 – Programming Languages

Programming Project – Technical Report

RPAL Interpreter in C++

Abeywardana S.M. – 230011P
Angeesa R.P.T – 230043N

1. Introduction

This report describes the design and implementation of an RPAL interpreter written in a single C++ source file (rpal20.cpp). The interpreter reads an RPAL source file and produces the correct program output, matching the reference rpal.exe.

The pipeline has five conceptual stages:

- Lexical Analysis – converts source text into tokens
- Parsing – builds an Abstract Syntax Tree (AST) from the token stream
- Standardization – transforms the AST into a Standardized Tree (ST) for display
- Evaluation – executes the AST directly using a tree-walking evaluator
- Output – prints the final computed value

The program is invoked as:

```
./rpal20 [-ast] [-st] <input_file>
```

The **-ast** flag prints the AST and exits; **-st** prints the Standardized Tree and exits.

2. Program Structure

The entire interpreter is implemented in a single file, rpal20.cpp, which uses the standard header <bits/stdc++.h>. All components are defined in the global namespace with no separate header files.

Prototype / Signature	Description
Lexer	Tokenises the source string into a vector<Token>
Parser	Recursive-descent parser; builds the AST as shared_ptr<Node> trees
standardize()	Free function; transforms the AST into a Standardized Tree (for -st display)
evalDef()	Evaluates definition nodes (=, fcn_form, and, within, rec) into bindings
eval()	Recursive tree-walking evaluator; dispatches on node label
apply()	Handles function application for CLOSURE, BUILTIN, and TUPLE values
main()	Reads input, orchestrates pipeline, handles -ast / -st flags

Execution order in main():

```
read file → Lexer::scan() → Parser::parse()
→ [standardize() if -st flag]
→ eval(ast, globalEnv) → print result
```

3. Lexical Analyzer

The lexer performs a single-pass character scan. It skips whitespace and `//` comments inline within the `scan()` loop. Two-character operators (`->`, `>=`, `<=`, `**`) are matched first via a priority list before single-character symbols.

Token structure

```
struct Token {
    string type;    // "ID", "INT", "STR", "SYM", "END"
    string text;
    int     line;
};
```

Lexer class

```
class Lexer {
    string s;
    size_t i = 0;
    int     line = 1;
    vector<string> twoOps = {"->", ">=", "<=", "**"};
public:
    explicit Lexer(string src);
    vector<Token> scan();    // main entry point
};
```

Prototype / Signature	Description
<code>explicit Lexer(string src)</code>	Initialises lexer with the source string
<code>vector<Token> scan()</code>	Single-pass scan; returns the full token sequence

Special handling:

- Whitespace – skipped via `isspace()`; newlines increment the line counter
- Comments – `//` single-line; consumed until end of line
- Identifiers – letters followed by alphanumerics or `_`; classified as ID
- Integers – digit sequences; classified as INT
- Strings – single-quoted with escape sequences `\t \n \\ \'`; unterminated strings are silently closed
- Two-character operators – `->`, `>=`, `<=`, `**` are tested before single-character fallback
- Single-character symbols – `(, ), ,, ;, .`, and operator characters become SYM tokens
- Unknown characters – throw `runtime_error` with line number

4. Parser

The parser uses recursive descent, one method per RPAL grammar non-terminal. Nodes are constructed with the free function `node()`, which produces `shared_ptr<Node>` objects.

Node structure

```
struct Node {
    string          label;
    vector<shared_ptr<Node>> ch;    // children
    explicit Node(string l = "");
};
using N = shared_ptr<Node>;

// factory
static N node(const string &label, vector<N> children = {});
```

Node labels encode both the kind and (for leaves) the value: `<ID:x>`, `<INT:42>`, `<STR:hello>`. Internal nodes carry operator or keyword labels such as `gamma`, `lambda`, `let`, `tau`, `->`, `aug`, etc.

Parser class

```
class Parser {
    vector<Token> t;
    size_t p = 0;
public:
    explicit Parser(vector<Token> toks);
    N parse();
private:
    N E();    N Ew();    N T();    N Ta();    N Tc();
    N B();    N Bt();    N Bs();    N Bp();
    N A();    N At();    N Af();    N Ap();
    N R();    N Rn();
    N D();    N Da();    N Dr();    N Db();
    N Vb();
    // helpers
    bool is(const string &x) const;
    bool type(const string &x) const;
    Token get();
    void need(const string &x);
    bool reservedWord(const string &w) const;
    bool idToken() const;
    bool startsRn() const;
    bool startsVb() const;
};
```

Prototype / Signature	Description
<code>explicit Parser(vector<Token> toks)</code>	Initialises parser with the token list
<code>N parse()</code>	Entry point; calls <code>E()</code> and verifies <code>END</code> token
<code>N E()</code> / <code>N Ew()</code>	<code>E</code> , <code>Ew</code> - top-level expression; <code>let</code> , <code>fn</code> , <code>where</code>
<code>N T()</code> / <code>N Ta()</code> / <code>N Tc()</code>	<code>T</code> , <code>Ta</code> , <code>Tc</code> - tuples <code>(,)</code> , <code>aug</code> , <code>conditional (->)</code>
<code>N B()</code> / <code>N Bt()</code> / <code>N Bs()</code> / <code>N Bp()</code>	<code>B</code> , <code>Bt</code> , <code>Bs</code> , <code>Bp</code> - boolean <code>or/and/not</code> and relational ops
<code>N A()</code> / <code>N At()</code> / <code>N Af()</code> / <code>N Ap()</code>	<code>A</code> , <code>At</code> , <code>Af</code> , <code>Ap</code> - arithmetic <code>+/-/*//</code> , <code>**</code> and infix <code>@</code>
<code>N R()</code> / <code>N Rn()</code>	<code>R</code> , <code>Rn</code> - function application and atomic terms
<code>N D()</code> / <code>N Da()</code> / <code>N Dr()</code> / <code>N Db()</code>	<code>D</code> , <code>Da</code> , <code>Dr</code> , <code>Db</code> - definitions, <code>rec</code> , <code>within</code> , <code>and</code>
<code>N Vb()</code>	Variable bindings: identifier or parenthesised list
<code>bool is(x)</code>	True if current token text equals <code>x</code>
<code>Token get()</code>	Consumes and returns current token
<code>void need(x)</code>	Consumes if matches; throws on mismatch
<code>bool idToken()</code>	True if current token is <code>ID</code> and not a reserved word

5. Standardization (AST → ST)

Standardization is implemented as a single recursive free function rather than a class. It performs a bottom-up traversal: children are standardized first, then a rule is applied to the current node. The result is used exclusively for `-st` display; the evaluator works directly on the original AST.

```
N standardize(const N &root);
```

Prototype / Signature	Description
<code>N standardize(const N &root)</code>	Recursively standardizes AST; returns ST root node

Transformation rules applied:

Prototype / Signature	Description
<code>let X=E1 in E2</code>	<code>gamma(lambda(X, E2), E1)</code>

Prototype / Signature	Description
E where X=E1	gamma(lambda(X, E), E1)
X=E1 within Y=E2	Y = gamma(lambda(X, E2), E1)
fcn_form f v1 v2 = body	f = lambda(v1, lambda(v2, body))
lambda(x, y, ...) body	Nested single-arg lambdas
and (parallel defs)	= (, (x1...xn), tau(E1...En))
rec X = E	X = gamma(<ID:Y*>, lambda(X, E))

6. Evaluator

The evaluator is implemented as a set of free functions rather than a class. It interprets the original AST (not the Standardized Tree) via the recursive eval() function with label-based dispatch. Definitions are handled separately by evalDef().

Value

```
struct Value {
    enum Kind { INT, STR, BOOL, TUPLE, CLOSURE, BUILTIN, NIL, DUMMY } kind;
    long long      i;          // INT
    string         s;          // STR
    bool           b;          // BOOL
    vector<Value>   tuple;      // TUPLE
    shared_ptr<Closure> clo;    // CLOSURE
    shared_ptr<Builtin> bi;     // BUILTIN
};
```

Factory helpers: makeInt(), makeStr(), makeBool(), makeTuple(), makeNil(), makeDummy(), makeBuiltin(), makeClosure().

Closure

```
struct Closure {
    vector<N> params;    // one or more parameter nodes
    N         body;
    EnvPtr     env;      // defining environment
};
```

Multi-parameter closures are supported natively: apply() pops the first parameter and returns a partially-applied closure when more remain, rather than nesting single-argument lambdas.

Builtin

```
struct Builtin {  
    string      name;  
    vector<Value> args; // accumulated arguments (currying)  
};
```

Environment

```
struct Env {  
    unordered_map<string, Value> m;  
    EnvPtr parent;  
    explicit Env(EnvPtr parent = nullptr);  
};
```

Prototype / Signature	Description
Value eval(N x, EnvPtr env)	Recursive tree-walking evaluator; dispatches on node label
Value apply(Value fn, Value arg)	Handles application for CLOSURE, BUILTIN, and TUPLE values
unordered_map<string, Value> evalDef(N d, EnvPtr env)	Evaluates definition nodes; returns name→value bindings
void bindPattern(EnvPtr, N pat, Value)	Binds a single ID or comma-tuple pattern to a value
Value lookup(EnvPtr, string name)	Walks parent chain; resolves built-in names if not in scope
string show(const Value &v)	Converts any Value to its printable string representation
bool equalVal(const Value &a, Value &b)	Structural equality for eq/ne operators

eval() dispatch

Prototype / Signature	Description
<INT:n>	Return makeInt from embedded text
<STR:s>	Return makeStr from embedded text
<ID:x>	Look up in environment via lookup(); built-ins resolved there
true / false	Return makeBool(true/false)
nil / dummy	Return makeNil() / makeDummy()
lambda	Package all-but-last children as params, last as body → CLOSURE

Prototype / Signature	Description
gamma	eval both children, call apply()
let / where	Create child env, call evalDef() for bindings, eval body
->	eval condition; branch to child[1] or child[2]
tau	eval all children into a TUPLE
neg / not	Arithmetic negation / boolean negation
or / &	Short-circuit boolean operators
aug	Append to TUPLE (or wrap scalar into new TUPLE)
+ - * / **	Integer arithmetic; ** via loop multiplication
eq / ne	Structural equality via equalVal()
gr / ge / ls / le	Integer or string comparison
@	Infix application: gamma(gamma(f, E1), E2)

evalDef() dispatch

Prototype / Signature	Description
=	eval RHS; bind to LHS (single ID or comma-tuple pattern)
fcn_form	Build CLOSURE from params and body; bind to function name
and	Process each sub-definition in the same parent env
within	Process first definition; extend env; process second definition
rec	Create a self-referential env; bind name before evaluating body (cyclic closure for functions; dummy-then-update for values)

apply() dispatch

Prototype / Signature	Description
CLOSURE	Create child env, bind first param, return partial closure or eval body
BUILTIN	Accumulate args; dispatch when arity is met (currying for Conc)
TUPLE	Integer index selection; 1-based

Built-in functions

Prototype / Signature	Description
Print	Print value to stdout; returns DUMMY
ItoS	Integer to string conversion
Order	Length of a TUPLE
Stem	First character of a string
Stern	Tail (all but first character) of a string
Null	True if NIL or empty TUPLE
Isinteger	Type predicate
Isstring	Type predicate
Istuple	Type predicate
Istruthvalue	Type predicate
Isfunction	True if CLOSURE or BUILTIN
Isdummy	Type predicate
Conc	String concatenation (two-argument curried built-in)

7. Conclusion

The RPAL interpreter is implemented as a single self-contained C++ source file. All components lexer, parser, standardizer, and evaluator are defined in the global namespace with no external dependencies beyond the C++ standard library.

The evaluator works directly on the original AST using a set of free functions (eval, evalDef, apply), handling all RPAL constructs including lexical scoping, multi-parameter closures, recursion via self-referential environments, tuple operations, and the full set of RPAL built-in functions. The standardize () function is provided separately for AST-to-ST display via the -st command-line flag.